
SHRFLOW EMAIL ENGINE REFERENCE MANUAL

Analytics & Intelligence Command Center

An Exhaustive Engineering Guide covering Platform Telemetry, tracking Ingest, SMTP Bounce Science, and Machine Learning Diagnostics

Document Classification: System Architecture Reference Guide

Release Version: 1.0.0 (Stable Production)

Release Date: May 2026

Prepared For: ShrFlow Technical Teams & Workspace Administrators

Copyright: © 2026 ShrFlow. All rights reserved.

1. Executive Summary & System Overview

The ShrFlow Analytics & Intelligence Command Center is the operational telemetry hub of the ShrFlow email system. Modern enterprise communication requires more than simple batch sending; it demands deep statistical visibility, real-time monitoring, and automatic deliverability protection to ensure that sent emails reach the recipient inboxes. This document details the software design, telemetry ingestion pipelines, database schemas, and AI diagnostic components that power ShrFlow's analytics engine.

1.1 System Objectives

The telemetry suite is engineered to fulfill three principal system goals:

- **Absolute Data Consistency:** Ensuring that every single email dispatch, pixel load, link redirect, and bounce notice is logged with high transactional integrity.
- **Low-Latency Event Ingestion:** Decoupling event tracking routes from core analytics dashboards to prevent web traffic load from stalling system performance.
- **Proactive Deliverability Monitoring:** Calculating bounce and spam rate percentages dynamically, providing workspace administrators with early warnings about list health and DNS status.

1.2 Real-World Value & Use Cases

In the email marketing industry, list decay and bad sender reputations are the leading causes of delivery failures. For example, sending campaigns to outdated contact lists containing invalid domains results in hard bounces. If a tenant's bounce rate exceeds 5%, mailbox providers (such as Gmail, Yahoo, and Outlook) start redirecting subsequent dispatches directly to the spam folder. By aggregating these metrics in real-time, ShrFlow enables administrators to identify problems early, configure SPF/DKIM keys, purge inactive contacts, and maximize ROI on active campaigns.

2. Core Technology Stack & Integration Design

The telemetry engine is built using modern frameworks optimized for throughput, security, and responsive layouts. By leveraging containerized services, the platform ensures rapid deployments and isolated service scaling.

2.1 Technology Stack Configuration

System Layer	Technology Stack	Core Role & Functionality
Backend API	FastAPI (Python 3.11)	Exposes protected telemetry endpoints, computes smart insights, parses chatbot queries, and handles JWT checks.
Database	PostgreSQL (Supabase)	Stores relational campaign data, dispatch queues, tracking events, and user profiles with custom indexing.
Frontend UI	Next.js 14 & React 18	Renders responsive control panels, dynamic 7d activity charts, and color-scaled engagement heatmaps.
Task Workers	RabbitMQ & Redis	RabbitMQ acts as the message broker, queuing campaigns for execution. Redis maintains lock controls.

2.2 Multi-Tenant Data Isolation Security Model

A foundational design pattern in ShrFlow is absolute isolation between organizations. Every incoming HTTP request must include a valid JSON Web Token (JWT) in the Authorization header. The FastAPI middleware extracts the token, verifies the cryptographic signature using a system secret, and decodes the identity context (user_id, role, tenant_id).

When querying telemetry datasets (such as campaigns, dispatches, and email events), the API injects the verified tenant_id directly into the Supabase PostgREST client: ``eq('tenant_id', tenant_id)``. This query isolation prevents any cross-tenant data leakage.

3. The Ingestion Pipeline & Campaign Preparation

To deliver accurate, live interaction statistics, the platform captures every lifecycle transition of an outbound email. The tracking workflow begins before the message is ever sent, during the template pre-compilation phase.

3.1 Pre-Compilation and Dynamic Personalization

When a campaign is triggered for dispatch, the worker process loads the base HTML template and retrieves the targeted audience contact records. For each contact, the engine compiles a personalized version of the email, replacing custom merge tags like `{{first_name}}` and `{{last_name}}` with the contact's actual profile data.

3.2 Injection of Tracking Codes

During this compilation step, the worker inserts tracking hooks. It appends an invisible image pixel for open tracking, and parses the HTML body to extract all hyperlinked anchor tags. Each standard link is replaced with a custom-generated tracking wrapper link unique to that recipient.

3.3 RabbitMQ Decoupling & Database Records

Sending thousands of emails simultaneously can create intense traffic spikes. To handle this, the backend decouples campaign launching from SMTP transmission. The API creates a Campaign row in PostgreSQL and generates 'PENDING' rows in the `campaign_dispatch` table for every contact. It then publishes these dispatch task IDs to a dedicated RabbitMQ queue.

Worker processes consume tasks from the queue at a controlled rate, format the SMTP payloads, connect to the delivery relay, and fire the messages. Once delivered, the worker updates the row status in `campaign_dispatch` to 'DISPATCHED'. If the SMTP handshake fails immediately, the status is set to 'FAILED' and the error details are logged.

4. Open Tracking Telemetry & Bot Protection

Tracking when an email is opened is critical for measuring subscriber interest. This is achieved using an invisible asset request, which must be carefully filtered to exclude automated traffic.

4.1 Open-Tracking Pixel Mechanics

During pre-compilation, the email sender worker appends an invisible 1x1 transparent PNG image at the bottom of the email body, just before the closing body tag. The image sources a tracking endpoint:

```

```

The 't' parameter is a lightweight JWT token containing the campaign_id, subscriber_id, and tenant_id, signed cryptographically by the platform. When the subscriber opens the email, their email client fetches this image, triggering a request to the FastAPI server. The API decodes the token, verifies its signature, and logs an 'open' event in the email_events database.

4.2 Bot Filtering and Data Accuracy

Many mailbox providers (especially Apple Mail and Gmail) cache email assets automatically on their proxy servers immediately upon delivery to protect users. This triggers a false 'open' request. To prevent these false events from skewing analytics, the API monitors incoming requests for proxy characteristics: checking user-agent strings (e.g., AppleNews, GoogleImageProxy) and tracking rapid multiple requests.

Proxy requests are logged with is_bot = True. The platform's dashboard and reporting queries filter out these rows, displaying only verified human interactions.

4.3 Real-World Walkthrough

1. A marketing manager sends a newsletter to subscriber 'Vinay'.
2. The email client on Vinay's phone downloads the invisible tracking pixel.
3. The request hits the API. The user-agent is identified as a mobile browser (human).
4. The API records a human 'open' event in the database, updating the campaign's open rate on the dashboard in real-time.

5. Click Telemetry & Redirect Architecture

Click-through rates are the primary indicator of subscriber engagement. Tracking clicks requires intercepting hyperlink navigations, registering the event, and redirecting the subscriber to their destination with minimal latency.

5.1 Anchor Link Rewriting Pipeline

When an email template is compiled, the system's HTML parser extracts all hyperlink href values. Each link is rewritten to route through the tracking gateway. For example, if the template contains a link to <https://yoursite.com/promo>, it is rewritten to:

```
<a  
href="http://api.shrflow.com/tracking/click?t=[JWT_TOKEN]&url;=https://yoursite.com/promo">Shop  
Now</a>
```

The JWT contains the subscriber's ID, the campaign ID, and the tenant ID, ensuring the click can be attributed to the exact contact.

5.2 Redirect API Handler

When the subscriber clicks the link, the browser opens the tracking gateway. The FastAPI server processes the request:

1. **Token Parsing:** The API extracts and decrypts the JWT token, validating the signature.
2. **Event Registration:** The API inserts a click record into the `email_events` table with the target URL, timestamp, and device details.
3. **Redirection Response:** The API returns an HTTP 302 'Found' response with the Location header set to the original destination URL.

5.3 Real-World Walkthrough

1. A subscriber receives an email and clicks the 'Shop Now' button.
2. The browser request goes to the API: `GET /tracking/click?t=eyJ...&url;=https://yoursite.com/promo`.
3. The API logs a 'click' event for the campaign and subscriber in PostgreSQL.
4. The API sends a 302 redirect. The subscriber's browser immediately loads the promo page. This entire redirection process takes less than 50 milliseconds, providing a seamless user experience.

6. Email Bounces & Deliverability Science

Deliverability represents the percentage of emails that successfully land in subscriber inboxes. Managing deliverability requires capturing and categorizing bounces—delivery failures reported by recipient servers.

6.1 Hard Bounces vs. Soft Bounces

Bounces are categorized into two types based on the permanence of the delivery failure:

- **Hard Bounces (5xx SMTP Codes):** Permanent delivery failures. These occur when an email address does not exist, the domain is invalid, or the recipient server has blocked the sender. Examples include SMTP error 550 (User Unknown). Hard bounces are critical deliverability threats and must be managed immediately.
- **Soft Bounces (4xx SMTP Codes):** Temporary delivery failures. These occur when a recipient's inbox is full, their mail server is temporarily down, or the message exceeds file size limits. Examples include SMTP error 422 (Inbox Full). The system will retry sending to soft-bouncing addresses for up to 72 hours before marking them as failed.

6.2 Webhook Listening and Event Processing

When an email bounces, the recipient's mail server sends a bounce notification to the SMTP delivery relay. The relay processes this bounce and fires a POST webhook request to the platform's webhook endpoint `/webhooks/delivery``. The FastAPI server parses this webhook payload, extracts the bounce code and description, and inserts a 'bounce' event row into `email_events`, linking it to the campaign and contact.

6.3 Suppression List Automation

To protect sender reputation, hard-bouncing contacts must never be emailed again. When a 'bounce' event is recorded, a database trigger or worker process updates the contact's status in the `contacts` table to 'bounced' or 'unsubscribed'. Future campaign queries filter out these contacts automatically: `.eq('status', 'active')``.

7. Subscriber Intelligence & Engagement Segmentation

A healthy email list is segmented by engagement. The Subscriber Intelligence engine groups the contact base dynamically based on their interaction history, helping administrators target active users and re-engage inactive ones.

7.1 Real-Time Segmentation Tiers

The backend route `/subscriber-intelligence`` scans the database to group subscribers into three engagement tiers:

Engagement Tier	Evaluation Rule	Recommended Strategy
Highly Engaged	Contact has ≥ 3 recorded open/click events	Target with promotional offers, reviews, and loyalty rewards.
Moderately Engaged	Contact has 1 or 2 open/click events	Send regular newsletters and surveys to build brand affinity.
Inactive List	Contact has 0 open/click events in database	Run a win-back re-engagement campaign, or purge to protect reputation.

7.2 Real-World Segmentation Example

Imagine an e-commerce brand launching a new product line. Instead of blasting the entire database—which increases spam complaints and list fatigue—the administrator filters the contacts to select only the 'Highly Engaged' segment. This segment has a proven track record of opening and clicking campaigns. As a result, the campaign achieves a high open rate (e.g., 45%) and a strong click-through rate, reinforcing positive sender reputation metrics with major inbox providers.

8. Optimal Send Time Heatmap (7x24 Matrix)

Email open rates are highly dependent on the day and time of delivery. To optimize this, the platform provides an interactive engagement heatmap visualizing when a tenant's subscribers are most active.

8.1 Data Gathering & Ingestion

The backend route ``GET /analytics/heatmap`` queries the `email_events` table for all open and click events matching the active tenant. Only events where `is_bot = False` are selected to ensure proxy hits do not corrupt the data. The query returns the `created_at` timestamp for each interaction event.

8.2 Weekday and Hour Binning Logic

For each timestamp, the API extracts the day of the week (0 to 6) and the hour of the day (0 to 23). The API accumulates counts in a 2D grid structure:

```
grid = {day: {hour: 0 for hour in range(24)} for day in range(7)}
```

This 2D grid is flattened into a JSON array of 168 coordinate objects: ``[{ 'day': day_idx, 'hour': hour_idx, 'value': count}]`` and returned to the frontend client.

8.3 Frontend Shading and Tooltips

The Next.js client renders the heatmap as a grid of 7 rows (Monday to Sunday) and 24 columns (hours 12 AM to 11 PM). The background opacity of each cell scales dynamically based on its count relative to the maximum count in the dataset: ``opacity = value / max_value``. A hover tooltip displays the exact day, hour, and event count for each cell, helping users visually identify the best times to schedule future campaign sends.

9. Predictive Analytics & Forecasting Models

The predictive analytics layer evaluates historical campaigns to forecast future performance, helping administrators set realistic benchmarks and improve their sending strategies.

9.1 Time-Series Predictive Forecasting

The `/predictive` endpoint projects expected open and click rates for the next campaign. The engine retrieves the last 10 campaigns sent by the tenant, calculates their individual open/click rates, and computes a weighted moving average. Recent campaigns are weighted more heavily than older ones, reflecting current subscriber interest and list hygiene trends.

9.2 Evaluating Trend Slope

The forecasting model compares the performance of the oldest campaign in the series against the most recent to determine the overall engagement trend. The slope is classified into one of three trends:

- **Upward Trend:** Average engagement is growing by $> 1\%$ (e.g., due to improving list quality).
- **Downward Trend:** Average engagement is dropping by $> 1\%$ (e.g., due to list fatigue or delivery issues).
- **Stable Trend:** Performance is flat, staying within a $\pm 1\%$ range.

9.3 Statistical Confidence Metrics

Forecasting accuracy depends on data volume. To reflect this, the model calculates a confidence score based on the number of sent campaigns: starting at 65% for a single campaign and scaling up to 92% once a history of 8 or more campaigns is established. This ensures administrators understand the statistical reliability of the predictions.

10. Conversational AI Analytics Chatbot

To make analytics accessible, the platform includes a floating chatbot assistant. Users can ask questions in plain English and receive real-time, database-driven answers.

10.1 Natural Language Query Parsing

The `/chat` endpoint accepts a text query from the user. Rather than running a static chatbot flow, the FastAPI backend parses the query to identify key intent tokens: open, click, bounce, contact, and campaign.

10.2 Dynamic Context Building

When an intent token is matched, the engine executes targeted database queries to pull live data for the active tenant. It then generates a natural-language response incorporating these live metrics, complete with optimization suggestions:

```
if 'open' in query:  
    return f"Your open rate is {open_rate}% with {num_opens} total opens. To improve this..."
```

10.3 Real-World Conversational Examples

• **User query:** 'What is my bounce rate?'

System response: 'You have recorded 12 total bounces/delivery failures, yielding a bounce rate of 1.2%. A healthy bounce rate should be under 2%. If it rises, you might need to clean your contact lists.'

• **User query:** 'Tell me about my contacts.'

System response: 'You currently have 1,420 total contacts in your audience database. You can segment them in the Subscriber Intelligence view based on their engagement history.'

• **User query:** 'Show me my account health.'

System response: 'Hello! Here is a snapshot of your live metrics: Contacts: 1,420; Campaigns: 15; Total Sent: 8,420; Open Rate: 22.4%; Click Rate: 3.1%; Bounce Rate: 1.2%. Ask me about any specific metric for details!'

11. Operations, Compliance, & Maintenance

Running a high-performance email analytics engine requires ongoing database maintenance, security controls, and compliance with international privacy laws.

11.1 Database Optimization & Indexing

As a workspace sends more campaigns, the `email_events` and `campaign_dispatch` tables can grow to millions of rows. To maintain fast query response times, the PostgreSQL database includes compound indexes on frequently queried columns: specifically `campaign_id + event_type` on `email_events`, and `campaign_id + status` on `campaign_dispatch`. This ensures that dashboard charts and AI chatbot queries resolve in milliseconds.

11.2 Delivery Compliance (SPF, DKIM, DMARC)

High deliverability rates require proper domain authentication. The platform monitors verified domains to ensure they have correct Sender Policy Framework (SPF), DomainKeys Identified Mail (DKIM), and Domain-based Message Authentication Reporting and Conformance (DMARC) records configured. If these records are missing or invalid, the smart insights engine flags a warning on the dashboard.

11.3 Privacy Regulations (GDPR & CAN-SPAM)

The platform is designed to comply with data privacy laws. Every marketing email must include an unsubscribe link. When a subscriber clicks the unsubscribe link, the tracking API logs the unsubscribe event and immediately updates the contact's status to 'unsubscribed' in the database. Future campaigns exclude these unsubscribed contacts automatically, ensuring compliance with CAN-SPAM and GDPR regulations.

Additionally, tracking tokens are encrypted and contain no raw personally identifiable information (PII), protecting subscriber privacy in transit.